Does positional encoding matter more than seed noise?

Paddy Davis · Single-GPU practitioner portfolio · Repo: github.com/davisp8-creator/Week-08-Career-Portfolio

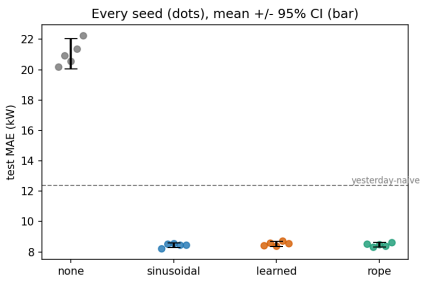
QUESTION AND BUDGET

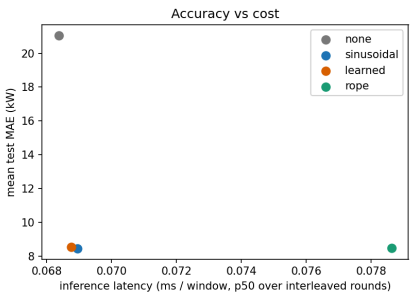
For a 69k-parameter Transformer forecasting the next 24 h of rural-feeder demand, does the positional encoding (none, sinusoidal, learned, RoPE) change test error by more than run-to-run noise? One free Colab T4, 4 arms × 5 seeds = 20 runs, about 8 minutes of wall-clock. Synthetic data; fp32; one factor changes.

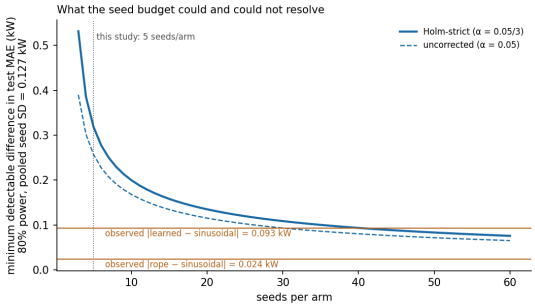
RESULT, WITH ITS BOUNDS

Arm (T4, 5 seeds)	Test MAE, kW	Δ vs sinusoidal, 95% CI	p50 latency, ms/window	Train s/run	Peak GPU MB
none	21.06 ± 0.80	+12.61 [+11.62, +13.60]	0.068	22.5	528
sinusoidal (ref.)	8.45 ± 0.13	–	0.069	22.7	528
learned	8.54 ± 0.13	+0.09 [−0.10, +0.29]	0.069	22.6	528
RoPE	8.47 ± 0.11	+0.02 [−0.16, +0.20]	0.079	26.7	570

- **Position information matters:** removing it costs 12.6 kW (about 22 seed SDs) and loses to a repeat-yesterday baseline (12.37 kW). Holm-adjusted p = 7e-6.
- **The other three tie, but a tie is not equality.** Five seeds could only detect a true gap of about 0.26–0.32 kW (3–4%) at 80% power. Settling a 0.1 kW gap would take about 36 seeds per arm, roughly an hour on a T4.
- **RoPE costs more:** +14% latency, +18% training time, +42 MB on the T4 (+16% and +20% in the first session); +34% latency on CPU. Retrained 3 more times: per-run MAE bit-identical, latency spread at most 0.0004 ms.







Hardware: Colab Tesla T4 14.6 GB (sm_75), driver 580.82.07, CUDA 13.0 build, torch 2.11.0, Python 3.13.16; 2-vCPU host. Seq 168→24, batch 128 / 256 / 512 (train / latency / eval), 10 epochs.

LIMITATIONS TO DISCOUNT BY

- Synthetic data and one data seed: error bars exclude data variance; 8.4 kW is not field accuracy.
- CPU and T4 runs differ in Python, torch, numpy and pandas as well as hardware. T4 repeats ran back to back on one VM; latency moved 2.4% between two sessions and no other GPU was tried.
- One tiny model, probably launch-bound and unprofiled; the cost ranking may flip at scale. Hyperparameters were not tuned per arm.

WHY THIS MATTERS HERE

Target: ML engineer on a four-person analytics team at a rural electric cooperative of about 40,000 meters, or at a 30-person vendor that sells day-ahead forecasts to such co-ops (hypothetical employer profile): no cluster, a day-ahead procurement desk, and a board and regulator who ask what a change costs and how sure you are. This package prices options in seconds and megabytes, says what could not be resolved, and ships documentation in EU AI Act Annex IV order with NIST AI RMF and ISO/IEC 42001 mappings.

SKILLS DEMONSTRATED → EVIDENCE

- Controlled ablation and seed variation → `configs/ablation.yaml`, `results/runs_t4gpu.csv`
- Honest benchmarking and memory budgeting → interleaved latency rounds, median-over-seeds peak memory, hardware manifest
- Compute reasoning → `scripts/power_analysis.py` (minimum detectable difference, seed cost in T4-minutes)
- Reproducible packaging and self-audit → pinned specs for both stacks, data SHA-256, resume-from-raw entry point, `AUDIT.md` (8 findings against my own earlier work)